

Lekcja 15 - DevOps w Databricks

2026-09

Data Engineering Course

DABs + DLT + API workflows

■ Agenda

- 1) Dlaczego klikany pipeline nie skaluje zespołu
- 2) DABs jako IaC dla Databricks
- 3) DLT jako deklaracyjny pipeline
- 4) CI/CD i bezpieczeństwo release

■ Problem

Pipeline "zyje" tylko w UI.

```
Brak historii zmian  
Brak reprodukowalnosci  
Rozjazd dev/prod
```

To klasyczny brak IaC i release procesu.

■ Cel operacyjny

Jeden kod -> wiele srodowisk
Powtarzalny deploy
Automatyczna walidacja przed prod

Databricks Asset Bundles:

```
jobs, workflows, config jako kod w repo
```

Daje review przez PR i kontrolowany deploy.

■ Struktura bundle

```
databricks.yml  
resources/  
src/
```

Konfiguracja i kod ida razem, nie osobno.

```
databricks bundle validate  
databricks bundle deploy --target dev  
databricks bundle run <job>  
databricks bundle deploy --target prod
```

■ Targety srodowisk

```
dev - szybki feedback  
test - stabilizacja  
prod - kontrolowany release
```

Kazdy target ma osobne zmienne i policy.

Delta Live Tables:

```
deklarujesz tabele i zaleznosci  
bronze -> silver -> gold
```

Silnik dba o orchestration i lineage.

■ DLT quality contracts

```
expect  
expect_or_drop  
expect_or_fail
```

Jakosc danych jest czescia definicji pipeline.

■ API-driven workflows

Integracja z zewnątrz:

```
jobs/run-now przez REST API
```

Przykład:

```
Airflow triggeruje Databricks przy event
```

■ CI/CD pattern

```
git push
-> validate
-> unit/integration tests
-> deploy dev
-> smoke test
-> approval
-> deploy prod
```

■ Promotion model

```
ten sam artifact  
promowany miedzy srodowiskami
```

Unikaj "osobnych" deployow tworzonych recznie.

■ Secrets i credentials

Zasada:

```
TAK: CI secrets, secret scopes  
NIE: tokeny w kodzie i repo
```

Sekret ma być podawany w runtime.

■ Observability release

Po wdrozeniu sprawdź:

- status jobs
- runtime vs baseline
- data quality metrics
- lineage i alerty

■ Typowe puapki

```
"szybka poprawka w UI"  
"dev i prod z inna konfiguracja"  
"brak testu smoke po deploy"
```

To prowadzi do dryfu i nocnych incydentow.

- 1) zatrzymaj promotion do prod
- 2) sprawdź validate + test logs
- 3) rollback do poprzedniego bundle version
- 4) uruchom smoke test i quality checks

Pytanie:

How do you productionize Databricks pipelines?

Odpowiedz:

I manage Databricks resources as code with Asset Bundles.
CI validates configuration, runs tests, and deploys by environment targets.
I use DLT for declarative dependencies and built-in quality expectations.
Secrets are injected at runtime from secure stores, never committed.

```
[ ] databricks.yml z targetami dev/prod  
[ ] pipeline DLT z co najmniej 1 expectation  
[ ] opis CI/CD promotion flow  
[ ] secrets strategy opisana  
[ ] smoke test po deploy
```

Oddajesz:

```
databricks.yml  
dlt_pipeline.py  
devops_flow.md
```

Najwazniejsze:

```
pokazac reprodukowalny i bezpieczny release  
pipeline
```

